



Dedication goes here.

1.5 Additional content can be written here.

*Acknowledgements

Acknowledgements go here.

ETSEIB

Contents

1	Introduction	5
2	Pure Diffusion Model	7
2.1	1D Transient Conduction Case	7
2.1.1	Solver Comparison	8
2.1.2	Numerical Parameters Study	8
2.1.3	Model Validation	10
2.2	2D Transient Conduction Case	10
2.2.1	Numerical Parameter Study	11
2.2.2	Model Validation	12
3	Convection-Diffusion Model	15
3.1	2D Convection	16
3.1.1	Numerical Parameter Study	17
3.1.2	Model Validation	17
4	2D Navier-Stokes Model	21
4.1	Fractional Step Method	21
4.2	Model Validation	22

Abstract

The objective of this project is to numerically solve the equations of fluid motion with heat transfer, i.e., the Navier-Stokes equations coupled with the energy conservation equation, in a multidimensional manner (CFD & HT methods). The methodology will be based on finite-volume techniques, using the fractional-step method to solve the coupling between the pressure and velocity fields, i.e., the conservation of mass and momentum equations. In a further step, the energy conservation equation will be incorporated, enabling the study of convection heat transfer problems. The analysis will begin with studies of conduction heat transfer problems (i.e., 4-materials problem). In a second step, a given velocity field will be incorporated to study the numerical diffusion problem in the solution of the convection-diffusion equation (i.e., Smith-Hutton problem). In the third step, a fluid motion solver for laminar flows will be developed, i.e., the mass and momentum conservation equations; finally, the energy conservation equation will be added to the fluid solver. The computational codes will be verified through grid-refinement studies in both space and time, using corresponding benchmark cases at each step. The investigation of the selected problems will include parametric studies considering variations in geometry, thermophysical properties, and boundary conditions.

Introduction

This report will cover the development of numerical tools for the resolution of Convection-Diffusion problems while consolidating the knowledge related to the transport phenomena, and the application of numerical methods for their resolution. The models implemented in this project will use a finite volume method to discretize the analyzed domain. In addition, the goal of this project implies the development of a robust and versatile code structure to ensure the models can be easily applied to any problem configuration. This chapter will introduce the main concepts related to the governing equations and the discretization method. Furthermore, the development of the models will be separated into a first portion focused on a pure diffusion model, and a second one implementing the complete convection-diffusion model. The code structure will be explained within each model implementation as a reflection of the evolution of the model throughout the development of this project.

When studying transport phenomena, there are two main mechanisms that govern the propagation of a transport variable ϕ . These are known as **Convection** and **Diffusion**, and classify the phenomena depending on what causes the transport of the transport variable ϕ . Diffusion refers to transport caused by molecular movement within the body, while Convection refers to transport caused by bulk movement of the body. The governing transport equation for this phenomena is given by Eq. 1, where each element can be discretized as shown in Eqs. 2, 3, 4. The discretization considers a rectangular prism control volume and is done considering a 2D mesh, keeping in mind that the 1D case ignores the elements for south and north neighbors. This discretization model will be the basis for the implementation shown in the following chapters, with appropriate clarifications for the nature of ϕ in each case.

$$\frac{\partial(\rho\phi)}{\partial t} + \nabla(\rho v\phi) = \nabla(\Gamma \nabla \phi) + S_\phi \quad (1)$$

$$\frac{\partial(\rho\phi)}{\partial t} = \rho \frac{\phi_P - \phi_P^0}{\Delta t} \quad (2)$$

$$\nabla(\rho v\phi) = \rho(v_E\phi_E S_E - v_W\phi_W S_W + v_N\phi_N S_N - v_S\phi_S S_S) \quad (3)$$

$$\nabla(\Gamma \nabla \phi) = \Gamma_W \frac{\phi_W - \phi_P}{d_{PW}} - \Gamma_E \frac{\phi_P - \phi_E}{d_{PE}} + \Gamma_S \frac{\phi_S - \phi_P}{d_{PS}} - \Gamma_N \frac{\phi_P - \phi_N}{d_{PN}} \quad (4)$$

1 Pure Diffusion Model

This chapter will explain the implementation of a pure diffusion model. For a pure diffusion model the governing transport equation can be reduced to the expression shown in Eq. 5, with the corresponding discretized version shown in Eq. 6. For the case of the thermal conduction model being implemented in this chapter, the equivalent expressions using Temperature as the transport variable and including the relevant physical parameters are shown in Eqs. (7), (8).

$$\frac{\partial(\rho\phi)}{\partial t} = \nabla(\Gamma\nabla\phi) + S_\phi \quad (5)$$

$$\rho \frac{\phi_P - \phi_P^0}{\Delta t} = \Gamma_W \frac{\phi_W - \phi_P}{d_{PW}} - \Gamma_E \frac{\phi_P - \phi_E}{d_{PE}} + \Gamma_S \frac{\phi_S - \phi_P}{d_{PS}} - \Gamma_N \frac{\phi_P - \phi_N}{d_{PN}} + S_\phi \quad (6)$$

$$\rho c_P V_P \frac{\partial T}{\partial t} = \nabla(\lambda \nabla T) + S_P \quad (7)$$

$$\rho c_P V_P \frac{T_P - T_P^0}{\Delta t} = \lambda_W S_W \frac{T_W - T_P}{d_{PW}} - \lambda_E S_E \frac{T_P - T_E}{d_{PE}} + \lambda_S S_S \frac{T_S - T_P}{d_{PS}} - \lambda_N S_N \frac{T_P - T_N}{d_{PN}} + S_\phi \quad (8)$$

1.1 1D Transient Conduction Case

In this section, the implementation of a C++ code for the resolution of a one-dimensional transient-state heat conduction model is described. The model works using a configuration file that includes multiple parameters that define the simulation. A summary of all the different parameters included in the file is shown in Table 1. As shown in the table, the configuration parameters have been separated into different groups. Using this model, a study will be made that analyzes the effect different parameters can have on resolution and performance. The main focus of this study will be to analyze the effect of some of the numerical parameters.

Physical includes the initial temperature and a list of the available materials and their thermophysical properties. **Geometrical** includes the geometric parameters; the model can only support rectangular shapes in this implementation and changes will be made when more complex geometries are studied. Sections must be defined with a material, the number of nodes for that specific section N_s , beginning and ending coordinates x_0 and x_1 , and internal heat generation q_V . **Numerical** includes the numerical parameters; the model currently has both the Gauss-Seidel (GS) and Conjugate Gradient (CG) solvers implemented and both will be compared as a first step before performing the relevant studies discussed ahead. **Mesh** includes the option to choose from four different algorithms, as well as all relevant parameters for each of these alternatives. **Boundary Conditions** includes an array describing the boundaries for the model and can currently support Dirichlet, Neumann, and Convection boundary conditions. In addition, it supports variable temperatures for Dirichlet boundary conditions.

Group	Parameter	Description
Physical Data	T0 materials	Initial value for temperature map. List of vectors with thermophysical properties corresponding to each material.
Geometrical Data	width height sections	Dimension in secondary axis. Dimension in secondary axis. List of vectors indicating the distribution of materials in the simulation. Also includes amount of nodes and internal heat generation.
Numerical Data	scheme solver maxIterations endTime timeStep tolNumeric tolTemporal	Scheme for simulation. Solver algorithm for simulation. Limit of iterations per timestep. Last instant of the simulation. Time increment after each instant. Tolerance for numerical solver. Tolerance for steady-state convergence.
Mesh Data	meshAlgorithm strength centering kappa delta	Algorithm for mesh generation. Cubic algebraic mesh parameter. Cubic algebraic mesh parameter. Power law mesh parameter. Hyperbolic tangent mesh parameter.
Boundary Data	boundaries	List of vectors describing the boundary conditions on each side.

Table 1: Configuration file structure.

1.1.1 Solver Comparison

As mentioned above, the model described in this report has both GS and CG solvers implemented. Only one will be used to perform the numerical parameter studies shown below; as such, relevant metrics need to be determined to compare both solvers. For this purpose, the number of iterations and residues for each time step will be used to compare both solutions. The base simulation used in this study will follow the case described in *Exercise 1: 1D transient conduction case*, with the addition of an internal heat source for a better comparison with the steady-state analytical solution.

This simulation will use $N = 100$, $\Delta t = 0.5$ and a Crank-Nicolson scheme with a uniform mesh. The results for both cases are shown in Figure 1. In both metrics, it can be seen that CG outperforms GS by a considerable margin; converging faster and obtaining lower residue. As such, CG will be used as the linear solver for the following studies. In addition, the computation time was also slightly faster for CG compared to GS with a total of 0.60 and 0.77 seconds, respectively; although this is the least relevant comparison.

1.1.2 Numerical Parameters Study

For this test, the time step Δt and the number of nodes N are modified to analyze the number of iterations and compare them with the analytical solution. Both parameters will be independently swept across their respective range of values, while keeping the other parameter at a constant value. A summary of the values for each one of these parameters is shown in Table 2,

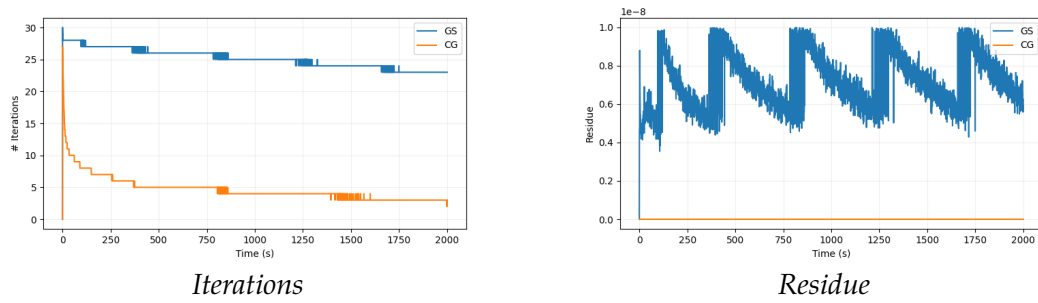


Figure 1: Solver comparisons.

highlighting the values used for the base case. In addition, the explicit scheme has an imposed limit for the value of Δt to ensure convergence, given by the expression shown in Eq. 9.

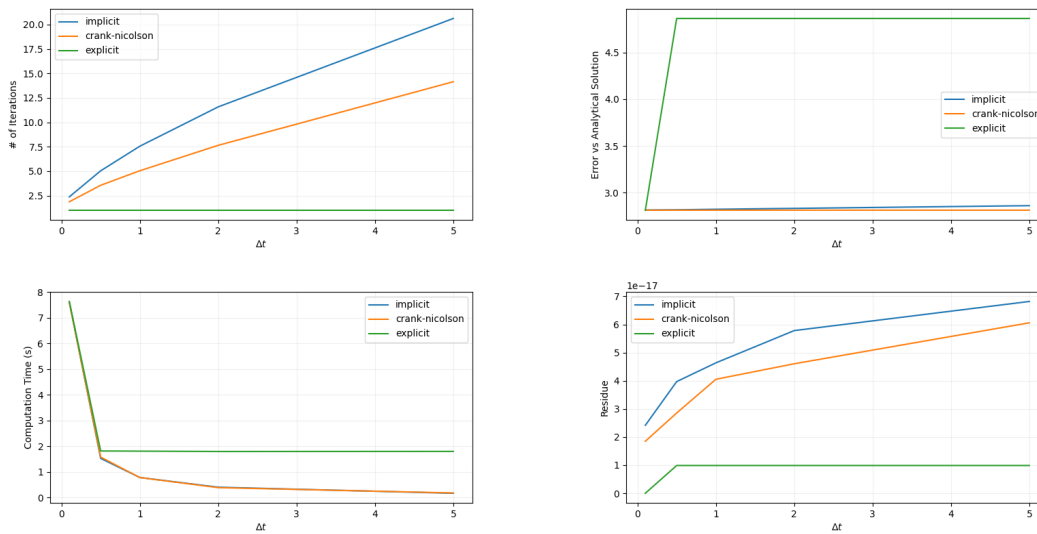
Parameter	Values
Δt	0.1, 0.5 , 1.0, 2.0, 5.0
N	50, 100 , 200, 400

Table 2: Detail of numerical parameters to be studied.

$$\Delta t_{max} = \frac{\Delta x^2}{2\alpha} \quad (9)$$

Δt Sweep The results obtained from the tests explained above are shown in Figure 2. As expected, the number of iterations remains unchanged for the explicit scheme, as it is meant to be a direct calculation. On the other hand, both the crank-nicolson and implicit schemes require more iterations per time step as it increases. Compared to the analytical solution, the explicit scheme shows a significant error for $0.1 < \Delta t$, while a small increase could be seen for the crank-nicolson and implicit schemes. Overall, the crank-nicolson scheme has shown better performance in terms of number of iterations needed and precision compared to the analytical solution. Furthermore, the results for explicit seem to stagnate at a constant value after the lowest time step value tested, suggesting that the limit set by Eq. 9 has been exceeded. However, the high error suggests that that expression should be revised for a more conservative estimation. The computation time is seen to be similar between the crank-nicolson and implicit schemes, with a small advantage for the implicit scheme.

N Sweep The results obtained for the tests explained above are shown in Figure 3. Similarly to the previous case, the explicit scheme is a direct calculation and does not require more iterations. In addition, showing an increase in the number of iterations for the crank-nicolson and implicit schemes, suggesting better performance for crank-nicolson as the mesh gets finer. Compared to the analytical solution, the explicit scheme still showed higher values compared to the crank-nicolson and implicit schemes; however, it seems to get closer as the number of nodes increases. In terms of computation time, both the crank-nicolson and implicit schemes saw a slight increase, while the explicit scheme saw a major difference as the mesh got finer.

Figure 2: Δt sweep results.

Case	Boundary Conditions
1	Neumann-Dirichlet
2	Convection-Dirichlet
3	Dirichlet-Neumann
4	Dirichlet - Convection
5	$T(t) = 100 + 50 \cos\left(\frac{0.01t}{5\pi}\right)$

Table 3: Cases for validation.

1.1.3 Model Validation

In addition to the Dirichlet boundary condition established in the problem statement, Neumann and Convection boundary conditions have also been implemented in the model. Furthermore, support for variable temperatures for Dirichlet boundary conditions was implemented. To validate this implementation, a comparison will be made with the analytical solution for a steady-state scenario with the transient-state model once it reaches equilibrium. A summary of the pairs of boundary conditions used for the tests is shown in Table 3, while comparisons with the steady state solution are shown in Figure 4. An additional case is included to test the proper implementation of the variable temperature Dirichlet boundary condition. As shown, all pairs evaluated accurately matched the analytical solution. Furthermore, the configuration file can now provide mathematical expressions instead of constant values when defining Dirichlet boundary conditions, as shown by the temperature profile. This capability can now easily be scaled up towards other functions in the model.

1.2 2D Transient Conduction Case

In this section, the implementation of a C++ code for the resolution of a two-dimensional transient state heat conduction model is described. This model has several modifications compared to the previous one, mainly reflected in the format for the configuration file. Table 4 summarizes the new parameters accepted in the configuration file. As shown, probes have now been

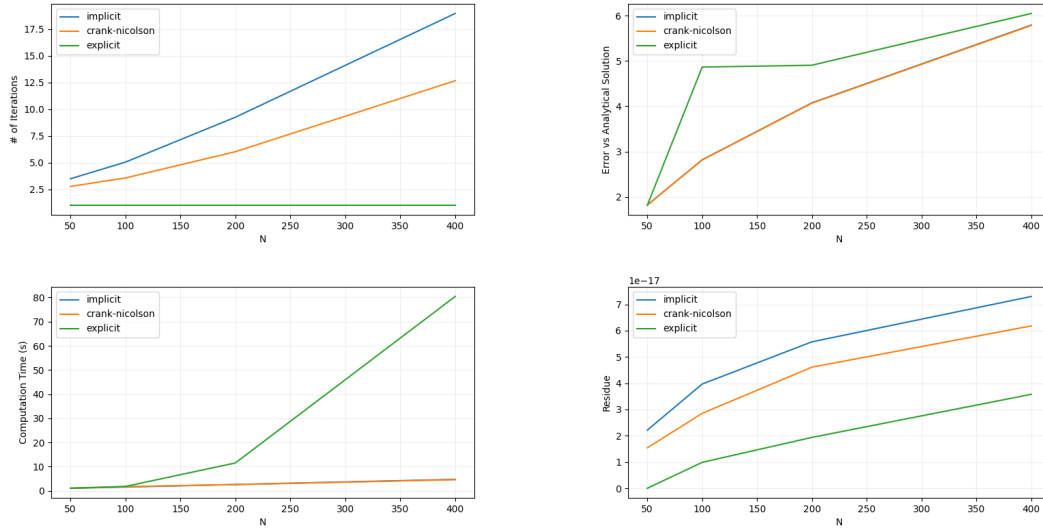


Figure 3: N sweep results.

added as a configurable parameter to allow the user to choose which data is being saved. In addition, refinement can now be defined independently for each axis and in separate regions to allow the user better control over the mesh. Furthermore, a Medic class was also included to handle diagnostics and validation of energy conservation in the system. This class is also used as the central tool for debugging as it can easily be modified to log any information relevant to the model.

1.2.1 Numerical Parameter Study

For this test, timestep Δt and number of nodes N are modified to analyze their impact on the number of iterations and total residue. Both parameters will be independently swept across their respective range of values while keeping the other parameter at a constant value. A summary of the values for each of these parameters is shown in Table 5, highlighting the values used for the base case. In addition, the explicit scheme still requires an imposed limit for the value of Δt , however the expression for the 2D case changes as described by Eq. 10. Contrary to the results shown for the 1D case, there will be no comparison of error with the analytical solution or computation time. This numerical parameter study will focus on comparing the number of iterations and total residue.

$$\Delta t_{max} = \frac{\Delta x^2 \Delta y^2}{2\alpha(\Delta x^2 + \Delta y^2)} \quad (10)$$

Δt Study The results obtained from the tests explained above are shown in Figure 5. As shown, the number of iterations and total residue follow a similar trend to the one seen in the 1D case, increasing with Δt for the implicit and crank-nicolson schemes. Showing a better performance for the crank-nicolson scheme on both metrics. On the other hand, the explicit scheme did not present any changes during the timestep study. This is due to the timestep being limited by the expression discussed above; which in this case happened for all analyzed cases.

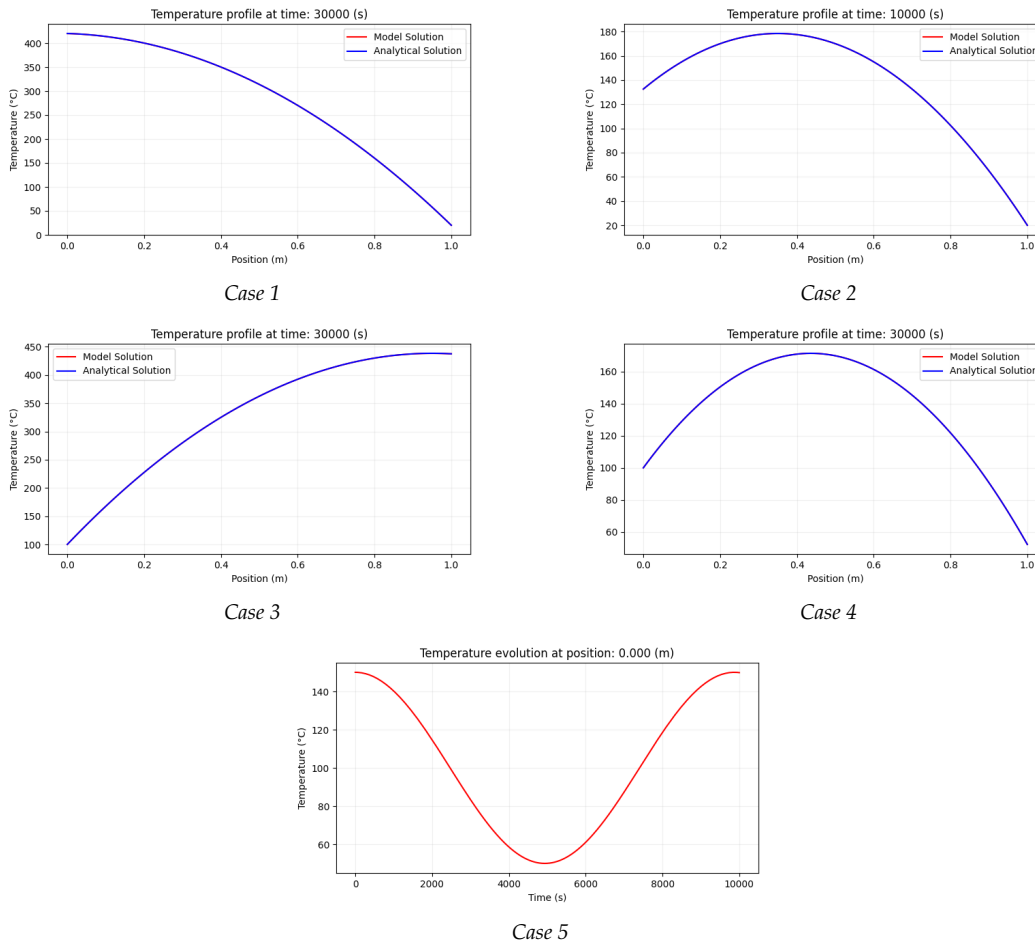


Figure 4: Comparison of model results with analytical solutions and implementation of mathematical expression reading for Dirichlet boundary condition.

N Study The results obtained from the tests explained above are shown in Figure 6. Just like in the previous test, the presented plots show a similar trend to the one seen in the previous scenario, showing an increase in number of iterations and residue for most cases. Furthermore, the implicit scheme showed higher sensitivity to increases in N compared to the crank-nicolson and explicit schemes; with the explicit scheme showing no change at all. On the other hand, while residue was significantly higher for implicit on coarser meshes, the difference between the implicit and crank-nicolson schemes became smaller as N increased; with the explicit scheme showing a slight increase in the beginning but less of an impact as N kept increasing.

1.2.2 Model Validation

In the same way as done in Section 2.1, boundary conditions were independently analyzed to validate proper implementation in the model by comparing it to the steady-state analytical solution. To simulate this behaviour, an adiabatic Neumann boundary condition ($\Delta\phi = 0$) was applied on the faces of the secondary axis in each test. On the other hand, the boundary conditions for the primary axis in the test were configured following the same testing plan described in Section 2.1. The results obtained in this validation successfully replicated the expected behaviour and produced the same plots as those seen in Figure 4 for each axis. Furthermore, a

Group	Parameter	Description
Numerical Data	scheme	Scheme selection for simulation.
	solver	Solver algorithm for simulation.
	tolNumeric	Tolerance for the numerical solver.
	tolTemporal	Tolerance for the steady-state convergence.
	maxIterations	Limit of iterations per timestep.
	endTime	Last instant of the simulation.
Physical Data	timeStep	Time increment after each instant.
	T0	Initial value for temperature map.
Geometrical Data	materials	List of vectors with thermophysical properties corresponding to each material.
	width	Dimension in the secondary axis.
Mesh Data	sections	List of vectors indicating the distribution of materials in the simulation. Also includes amount of nodes and internal heat generation.
	strength	Cubic algebraic mesh parameter.
	centering	Cubic algebraic mesh parameter.
	kappa	Power law mesh parameter.
	delta	Hyperbolic tangent mesh parameter.
	meshAlgorithm	Algorithm for mesh generation.
Boundary Data	N	List specifying total number of nodes for each axis.
	refinement	List of vectors specifying refinement configuration for each axis.
Probe Data	boundaries	List of vectors specifying the boundary conditions on each side.
	probes	List of vectors specifying the probes configured for the simulation.

Table 4: Configuration file structure.

comparison with the *Exercise 3: 4 Materials Case* will be used as a benchmark to complete the validation of the model.

4 Materials Case The results obtained for *Exercise 3: 4 Materials Case* are shown in Fig. 7. As mentioned above, this is a transient simulation and there is no analytical solution available to compare the results with. Instead, this case provides reference values for two separate points at different instants of the simulation. As seen in the results, the model was able to match the values provided for points $[0.65, 0.56]$ and $[0.74, 0.72]$; as well as showing a temperature distribution consistent with the provided time instants.

Parameter	Values
Δt	0.1, 0.5, 1.0 , 2.0, 5.0
N	50, 100 , 200, 400

Table 5: Value ranges for numerical parameter study.

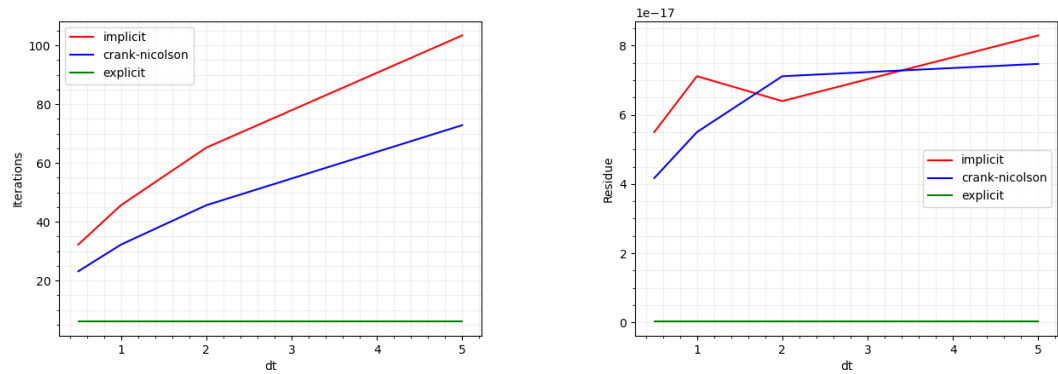


Figure 5: Δt sweep results.

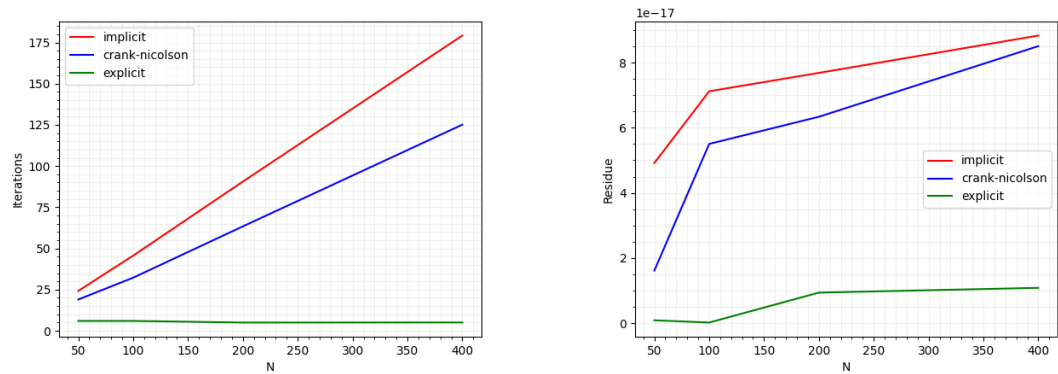


Figure 6: N sweep results.

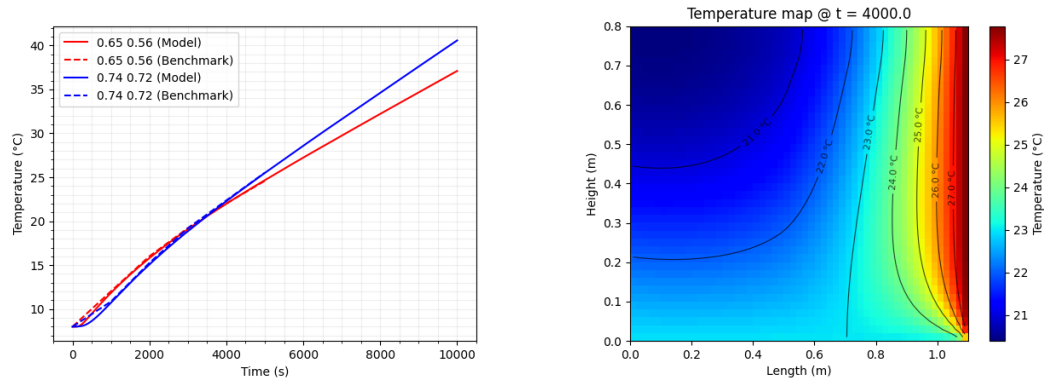


Figure 7: Comparison of model results with results provided in problem statement.

2 Convection-Diffusion Model

This chapter will explain the implementation of a convection-diffusion model. In this case, the complete transport equation shown in Chapter 1 needs to be implemented, considering both the diffusive and convective elements. The model implementation will be validated by solving the Smith-Hutton case and comparing the results to available reference values. In addition, the implementation described in this chapter will follow the discretization model described by Patankar [1], which proposes a generalized formulation that considers the various convective schemes available for numerical resolution. Based on said formulation, the discretized coefficients for a 2D model are shown below in Eqs. 11, 12, 13, 14, 15, 16.

$$a_W = D_w A(|P_w|) + ||F_w, 0|| \quad (11)$$

$$a_E = D_e A(|P_e|) + ||-F_e, 0|| \quad (12)$$

$$a_S = D_s A(|P_s|) + ||F_s, 0|| \quad (13)$$

$$a_N = D_n A(|P_n|) + ||-F_n, 0|| \quad (14)$$

$$a_P = a_W + a_E + a_S + a_N + a_P^0 \quad (15)$$

$$b_P = S_\phi V_P + a_P^0 \phi_P^0 \quad (16)$$

$$a_P^0 = \frac{\rho_P^0 V_P}{\Delta t}$$

As mentioned above, this discretization model includes the additional $A(|P|)$ term, which corresponds to a different function depending on the convective scheme. A summary of the expressions used for the convective schemes implemented in the model is shown in Table 6.

Scheme	$A(P)$
CDS	$1 - 0.5 P $
UDS	1
Hybrid	$ 0, 1 - 0.5 P $
Power Law	$ 0, (1 - 0.1 P)^5 $
Exponential	$\frac{ P }{e^{ P }-1}$

Table 6: The function $A(|P|)$ for different schemes [1].

2.1 2D Convection

This section will cover the implementation of a C++ code for the resolution of a two-dimensional transient-state convection-diffusion transport model. The model works using a configuration file that follows the same structure discussed in the previous chapter with some additional modifications implemented when implementing this section of the model. A summary of the parameters included in the file is shown in Table 7. As it can be seen, the main modifications were the addition of the velocity field, separation of temporal and convective scheme parameters, and the inclusion of a Medic data section designed to control the diagnostics tool. Furthermore, due to the asymmetrical nature of convective transport, an additional numerical solver using the Biconjugate Gradient Stabilized Method had to be implemented to ensure convergence and stability. Finally, the model has fundamentally changed from having defined nodes located at the boundaries to having ghost nodes. In consequence, a new Boundary probe type was introduced to store the values at the boundaries.

Group	Parameter	Description
Numerical Data	tempScheme	Temporal scheme selection for simulation.
	spatScheme	Convective scheme selection for simulation.
	solver	Solver algorithm for simulation.
	tolNumeric	Tolerance for the numerical solver.
	tolTemporal	Tolerance for the steady-state convergence.
	maxIterations	Limit of iterations per timestep.
	endTime	Last instant of the simulation.
Physical Data	timeStep	Time increment after each instant.
	T0	Initial value for temperature map.
	materials	List of vectors with thermophysical properties corresponding to each material.
Geometrical Data	velocityField	List of expressions for each component of the velocity field.
	width	Dimension in the secondary axis.
Mesh Data	sections	List of vectors indicating the distribution of materials in the simulation. Also includes amount of nodes and internal heat generation.
	strength	Cubic algebraic mesh parameter.
	centering	Cubic algebraic mesh parameter.
	kappa	Power law mesh parameter.
	delta	Hyperbolic tangent mesh parameter.
	meshAlgorithm	Algorithm for mesh generation.
	N	List specifying total number of nodes for each axis.
Boundary Data	refinement	List of vectors specifying refinement configuration for each axis.
	boundaries	List of vectors specifying the boundary conditions on each side.
	probes	List of vectors specifying the probes configured for the simulation.
Medic Data	medicOn	Control argument for diagnostic tool.

Table 7: Configuration file structure.

This model will be validated by solving the Smith-Hutton problem, being used both as the test case for the numerical parameter study and for the model validation. The model validation will be done by comparing the model results to reference values provided for cases with $\rho/\Gamma = 10^1, 10^3, 10^6$. The numerical parameter study will compare the number of iterations and total residue for each of these cases for two of the convective schemes implemented for the model; namely Central Difference Scheme (CDS) and Upwind Difference Scheme (UDS).

2.1.1 Numerical Parameter Study

The results obtained for the previously described numerical parameter study are shown in Fig. 8. As shown, using CDS will result in a higher number of iterations as the ratio increases, not being able to reach convergence within the simulation constraints for the case where $\rho/\Gamma = 10^6$. Figure 9 shows the obtained colormap on the last time-step of the simulation. As it can be seen, there is severe checkerboarding across the domain. On the other hand, UDS was able to converge for all values across the tested range. As such, model validation will only show the colormaps for UDS. These divergence is seen due to the Peclet number (Pe), which is a measure of dominance between Convection and Diffusion for a given transport phenomena where $Pe \ll 1$ means the phenomena is diffusion dominant and $Pe \gg 1$ means it is convection dominant. This means that CDS has a particular limitation that can be seen in its definition in Table 6. For simulations where $Pe > 2$, the expression will have an inverted sign and break the diagonal dominance requirement for the linear solver. On the other hand, since UDS only takes values from one side, the coefficients will never see a sign inversion, making it unconditionally stable.

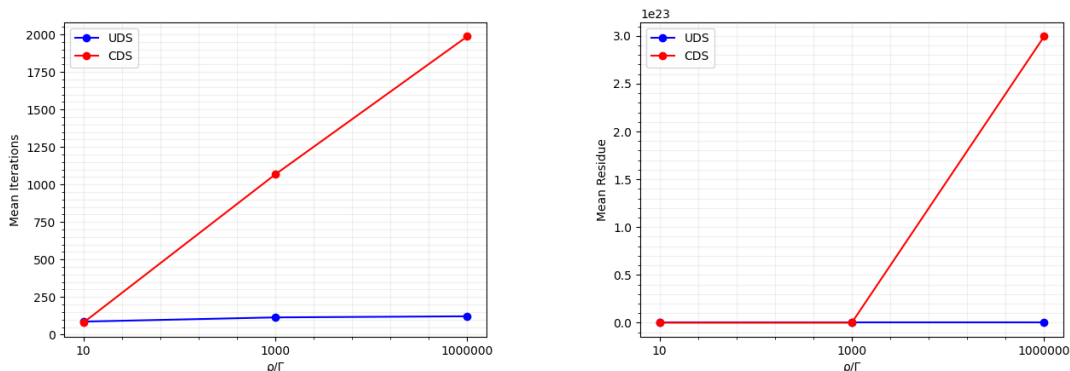


Figure 8: Convective scheme comparison.

2.1.2 Model Validation

As mentioned above, the model will be compared to the provided reference values for the Smith-Hutton problem, these values were obtained using CDS but this report will compare both UDS and CDS. This problem consists of a defined velocity field in a control volume where the presence of a transport variable is introduced. The validation will consist of comparing the steady-state solution at the boundaries with reference values for this particular problem in three main scenarios that consist of changing the ratio between the density ρ and the diffusive transport coefficient Γ . For this test, Γ will remain fixed at 1 while ρ will be evaluated at $10^1, 10^3$, and 10^6 . The results of this test cases are shown in Fig. 10; as it can be seen the, model accurately matched the expected results at the boundaries, with a slight expected deviation for UDS. Con-

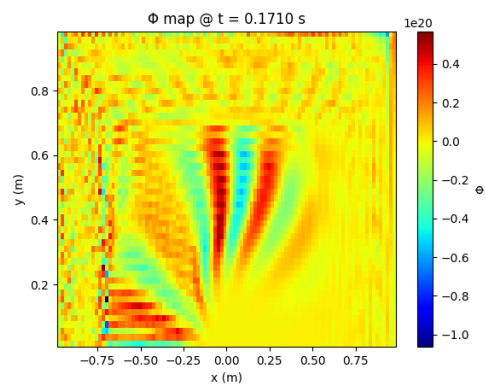


Figure 9: CDS divergence for $\rho/\Gamma = 10^6$

trary to CDS which is second-order accurate, UDS is first-order accurate, this implies certain mathematical simplifications and as such, a slight error against the exact solution. This effect is generally seen in the smoothening of sharp gradients as can be seen in these results. In addition, the obtained steady-state solutions for UDS simulations are shown in Figure 11.

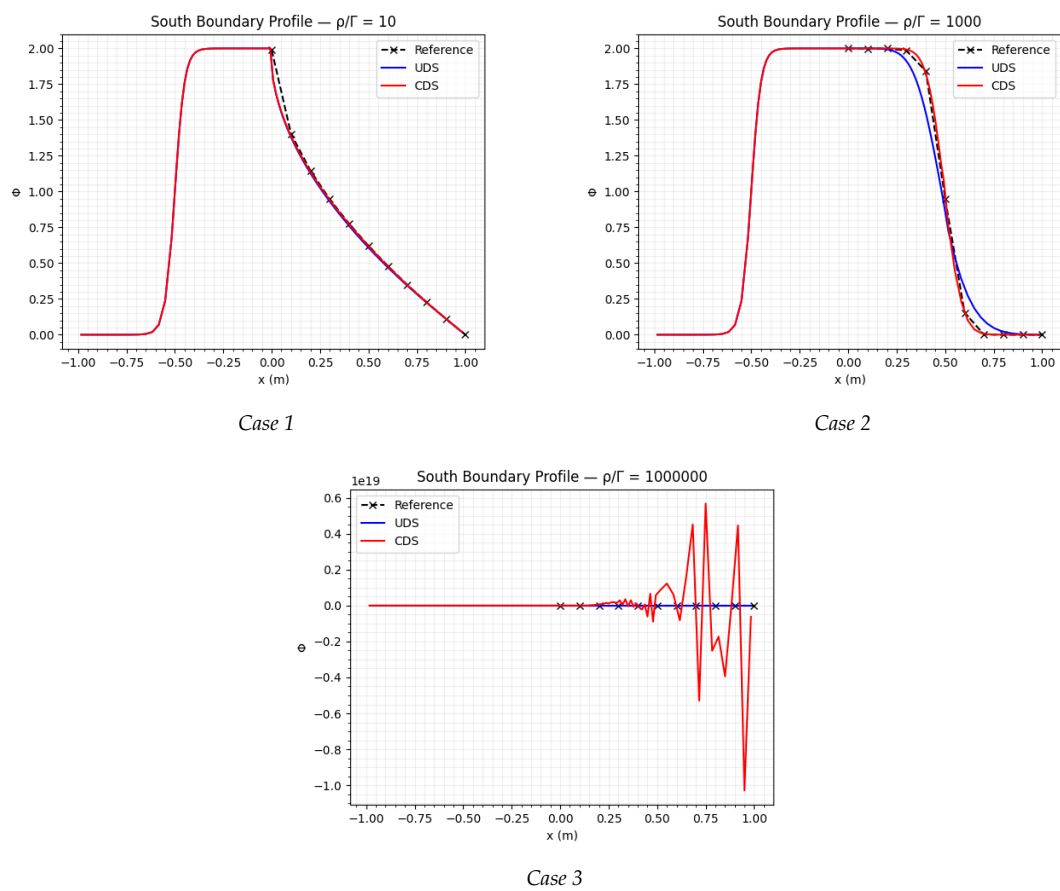


Figure 10: Comparison of model results with results provided in problem statement.

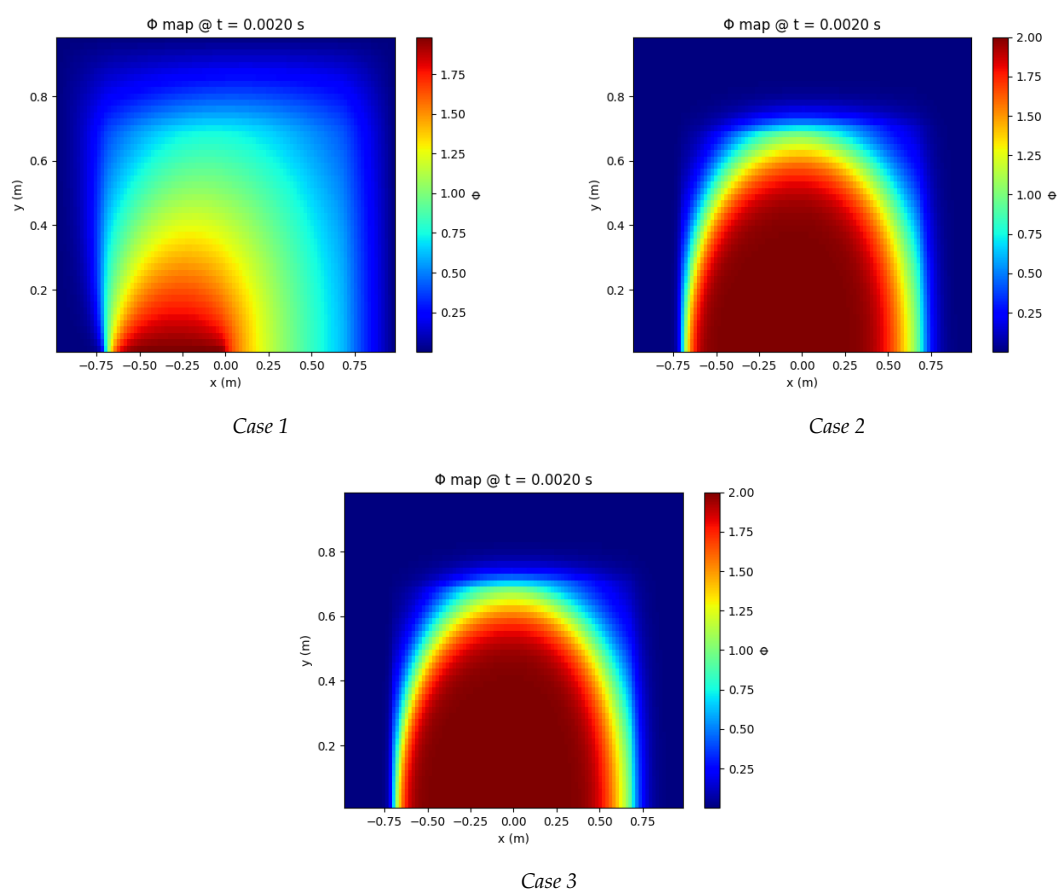


Figure 11: UDS results for all cases.

3 2D Navier-Stokes Model

This chapter will describe the implementation of a numerical solver for 2D Navier-Stokes problems, using the Fractional Step Method and doing the temporal discretization using an Adams-Bashforth method. The Navier-Stokes equations describe the movement of fluids by describing multiple characteristics of the movement in a system of equations. The equations shown in Eq. 17, 18, 19 describe the mass conservation, momentum conservation, and energy conservation respectively. As it can be seen, these equations follow the same structure behind transport equations shown in Chapter 1, with the corresponding accumulation, convective, diffusive, and source terms. The model will be evaluated against reference values provided for the Lid-Driven Case problem [?], which simplifies the model by removing the energy conservation evaluation. As such, the model will require only 3 equations, one for mass conservation, and the 2 geometric components of the momentum conservation.

$$\nabla \cdot \mathbf{v} = 0 \quad (17)$$

$$\rho \frac{\partial \mathbf{v}}{\partial t} + \rho(\mathbf{v} \cdot \nabla) \mathbf{v} = \mu \nabla^2 \mathbf{v} - \nabla P \quad (18)$$

$$\rho c_p \frac{\partial T}{\partial t} + \rho c_p \nabla(\mathbf{v} T) = \nabla(\lambda \nabla T) + \dot{q}_V \quad (19)$$

As mentioned above, this model uses the Adams-Bashforth discretization method, which relies on two previous time-steps to evaluate the next iteration. Eq. 20 shows the expression used for this discretization method, where $R(\mathbf{v})$ corresponds to the convective and diffusive terms evaluated at the corresponding time-step and is described by Eq. 21. The discretization for this expression will follow the discretization implemented in Chapter 3.

$$\rho \frac{\mathbf{v}^{n+1} - \mathbf{v}^n}{\Delta t} = -\nabla P^{n+1} + \frac{3}{2} R(\mathbf{v}^n) - \frac{1}{2} R(\mathbf{v}^{n-1}) \quad (20)$$

$$R(\mathbf{v}) = \mu \nabla^2 \mathbf{v} - \rho(\mathbf{v} \cdot \nabla) \mathbf{v} \quad (21)$$

3.1 Fractional Step Method

This section will explain the different modifications that were implemented in the model to solve the current test case. As mentioned above, there is a requirement to solve multiple transport variables at the same time that are mutually dependent. Since the velocity and pressure need to be computed simultaneously, the Fractional Step Method separates this process into three separate steps. The first step consists of evaluating the velocity field while ignoring the pressure field. This calculated velocity field will not be an accurate solution for the next time-step; however, it can be used to evaluate the pressure field at the next time-step. After that second step, the only remaining operation is to correct the calculated velocity field with the influence of the now known pressure field. The three steps can be summarized by the expressions shown in Eqs. 22, 23, 24, showcasing the methodology used to obtain the solution to this

particular case. Since the two velocity components are calculated explicitly, there is no need to employ a numerical solver for those variables. On the other hand, the pressure equation will have the same structure as a diffusion equation as it can be seen in Eq. 23.

$$\mathbf{v}^p = \mathbf{v}^n + \frac{\Delta t}{\rho} \left[\frac{3}{2} R(\mathbf{v}^n) - \frac{1}{2} R(\mathbf{v}^{n-1}) \right] \quad (22)$$

$$P_p = \frac{1}{a_p} (a_n P_N + a_s P_S + a_e P_E + a_w P_W - \frac{\rho \Delta S}{\Delta t} (u_e^p - u_w^p + v_n^p - v_s^p)) \quad (23)$$

$$\mathbf{v}_p = \mathbf{v}_p^p - \frac{\Delta t}{\rho} \nabla P^{n+1} \quad (24)$$

Since this is now a coupled solver solving for multiple variables instead of solving for a single one, the configuration file will now require the relevant details and information about every variable. As such, initial values and boundary conditions now need to be specified separately for the main solver variable and for the secondary, explicitly calculated, variables. In addition, a new Field probe was introduced specifically for the velocity field. Table 8 describes all the configuration parameters available for the current model.

3.2 Model Validation

As mentioned above, the model will be validated by comparing the results to given reference values for the Lid-Driven Case problem with $Re = 100, 400$ using both CDS and UDS. Re is defined by Eq. 25 No numerical study will be included in this section as the velocity components are calculated explicitly and the pressure equation is solved using the same model described in previous chapters. Figure ??

$$Re = \frac{\rho U L}{\mu} \quad (25)$$

PENDING PASTE PLOTS AND WRITE COMMENTS

Group	Parameter	Description
Numerical Data	tempScheme spatScheme solver tolNumeric tolTemporal maxIterations endTime timeStep	Temporal scheme selection for simulation. Convective scheme selection for simulation. Solver algorithm for simulation. Tolerance for the numerical solver. Tolerance for the steady-state convergence. Limit of iterations per timestep. Last instant of the simulation. Time increment after each instant.
Physical Data	PHI0 VF0 materials velocityField	Initial value for main solver variable. List with initial value for each component of velocity. List of vectors with thermophysical properties corresponding to each material. List of expressions for each component of the velocity field.
Geometrical Data	width sections	Dimension in the secondary axis. List of vectors indicating the distribution of materials in the simulation. Also includes amount of nodes and internal heat generation.
Mesh Data	strength centering kappa delta meshAlgorithm N refinement	Cubic algebraic mesh parameter. Cubic algebraic mesh parameter. Power law mesh parameter. Hyperbolic tangent mesh parameter. Algorithm for mesh generation. List specifying total number of nodes for each axis. List of vectors specifying refinement configuration for each axis.
Boundary Data	boundaries boundariesVelocity	List of vectors specifying the boundary conditions on each side for the main solver variable. List of vectors specifying the boundary conditions on each side for the velocity components.
Probe Data	probes	List of vectors specifying the probes configured for the simulation.
Medic Data	medicOn	Control argument for diagnostic tool.

Table 8: Configuration file structure.

Conclusions and Future Work

This document has covered the development and evolution of a Heat and Mass Transfer solver across different benchmarks of increasing complexity. Across the different stages of development of this model, different complexities regarding the implementation of numerical methods and the simplification of models have been observed and analyzed. This behaviour is consistent with the mathematical formulation behind the numerical solver and must be considered when configuring the simulation parameters. In addition, the model developed during this project will serve as a stepping stone towards more complex implementations. There are still several improvements that can be made to the model proposed with this project, particularly in terms of optimization of variables and some redundant algorithms. The model will continue to be maintained to expand its capacity and increase its performance, addressing all of the mentioned inefficiencies.

PENDING WRITE COMMENTS

1. Model 1D was simple but the structure of the code was archaic
2. Model 2D forced structure for the Mesh but discretization pretty much remained the same
3. Model Convection forced a modification of the discretization
4. Model NS was an extension of convection, including Adams-Bashforth and Fractional Step Method
5. Inefficiencies in the code (storing positions instead of indexes for boundaries, improved in the later versions, mention minor improvements in algorithms, vectors that remain empty instead of being smaller, variable types that could be standardized to consume less memory)

References

- [1] Suhas V. Patankar. *Numerical Heat Transfer and Fluid Flow*. Series in Computational Methods in Mechanics and Thermal Sciences. Hemisphere Publishing Corporation, Washington, D.C., 1980.